

Git and Github

Git – Version Control System

What is Git?

- **Git** is a **distributed version control system (VCS)** used to track changes in files (mostly source code).
- It allows multiple people to collaborate on the same project without overwriting each other's work.

Problems Before Git

1. Sharing Code

- Without VCS, developers had to share code via USB drives, email, or shared folders — messy and prone to overwriting.
- Git allows **central repositories** (GitHub, GitLab, Bitbucket) where everyone can push/pull changes.

2. Versioning

- Without versioning, overwriting changes is common.
- Example: If a function is unused, you might delete it, but later realize you need it — without version control, it's gone.
- Git keeps a **history of every change**, so you can restore deleted code anytime.

3. Tracking Changes

- Without Git, it's hard to know what exactly changed and when.
- Example: "What changes did I make 3 days ago?"
 - Git can **show commits** from that date with exact file changes.

How Git Solves These Problems

- **Collaboration:** Multiple developers can work in parallel using branches and merges.
- **History & Rollback:** Every change is recorded with timestamps, authors, and commit messages.

- **Branching:** Experiment on a branch without affecting main code.
- **Distributed:** Each developer has a full copy of the repo, so no single point of failure.

"Evolution of Version Control: CVS → SVN → Git"

- **CVS (Concurrent Versions System)** – Early centralized VCS, basic versioning, slow, poor merge handling.
- **Subversion (SVN)** – Improved CVS, still centralized, better branching but server-dependent.
- **Git** – Distributed, works offline, fast branching/merging, now the most popular.

Centralized VCS and Distributed VCS

Centralized VCS

- Single central server stores all code.
- Examples: CVS, Subversion (SVN).
- Need server connection to commit or view history.
- If server is down, work stops.
- When you pull (or checkout), you get **only the current snapshot** of the project files. You don't get the complete commit history unless you explicitly request it from the server.

Distributed VCS

- Every developer has a full copy of the repository.
- Examples: Git, Mercurial.
- Works offline; you can commit and view history locally.
- Server mainly used for collaboration and syncing.
- When you clone or pull for the first time, you get **the full repository + complete commit history**. That's why you can view logs, checkout old commits, and even work offline.

Feature	Centralized VCS (CVCS)	Distributed VCS (DVCS)
Examples	CVS, Subversion (SVN)	Git, Mercurial
Repository storage	Single central server	Every developer has full copy

Feature	Centralized VCS (CVCS)	Distributed VCS (DVCS)
Commit location	Only on central server	Local machine (offline possible)
Working offline	Limited (can't commit)	Full support (commit, view history)
Data retrieved on checkout/clone	Latest snapshot	Full history + snapshot
Server dependency	High — server must be up for commits	Low — needed only for collaboration/push
Risk of server failure	High — if central server is lost, all history is gone	Low — history exists on all copies

Git vs GitHub

Git

Git is a **distributed version control system (DVCS)** that tracks changes to files and enables multiple people to work on the same project simultaneously. It lets you create branches, merge changes, and maintain the complete history of a project locally without needing a central server.

GitHub

GitHub is a **cloud-based platform** that hosts Git repositories and provides collaboration features such as pull requests, code reviews, issue tracking, and project management tools. It builds on top of Git to make sharing and working on code with others easier.

-
- **Git** → Open-source software → Download and install locally.
 - **GitHub** → Online platform → Adds usability features like raising issues, commenting, reviewing PRs.
 - Similar platforms: **GitLab**, **Bitbucket**.

Feature	Git	GitHub
What it is	Distributed version control system (tool)	Web-based hosting service for Git repositories
Purpose	Track changes to files, manage versions locally & remotely	Store Git repos online, enable collaboration, issue tracking, code reviews
Where it runs	On local machine (CLI or GUI tools)	On the internet (cloud)
Works without internet?	Yes (commit, branch, merge offline)	No (needs internet for access)
Cost	Free, open-source	Free & paid plans
Ownership	Git community	Microsoft

Feature	Git	GitHub
Example use	<code>git commit -m "Fix bug"</code>	Create a Pull Request for code review

Example

```
mkdir example.com
```

```
cd example.com
```

```
ls
```

```
vim calculator.sh
```

```
# x = a + b
```

```
ls
```

Download Git:

```
https://git-scm.com/downloads
```

```
https://git-scm.com/downloads/linux
```

Once installed:

```
git # shows all git commands
```

Create git repository:

```
git init
```

```
# Initialized empty git repository
```

3 important commands every DevOps engineer should know (Git lifecycle):

```
git add .
```

```
git commit -m "Commit message"
```

```
git push
```

How to know git repository is created:

ls -la # shows hidden .git folder

.git folder is where git tracks everything in the project. If you delete this folder, git will stop tracking the repository.

ls .git

HEAD config hooks objects refs

.git folder details:

objects → stores file contents as git objects

hooks → scripts to automate checks (e.g., prevent committing passwords or API tokens)

config → git configuration like username, email, credentials

HEAD → points to the current branch

Create a sample file

echo "x = a + b" > calculator.sh

Check the status of files in the repo

git status

untracked file: git says "I don't know whether I should keep track of it or not"

nothing to track — use git add to start tracking

Add file to staging

git add calculator.sh

Now there are no untracked files

If someone changes the file and you run git status:

git status

modified: calculator.sh

Show exact changes made

git diff

Commit the changes

```
git commit -m "This is my 1st version"
```

Check status again

```
git status  
# nothing to commit, working tree clean
```

Add another line to the file

```
echo "x = a = b" >> calculator.sh
```

Check status and changes

```
git status  
git diff
```

Commit second version

```
git commit -m "Second version"
```

View commit history

```
git log  
git log --oneline
```

Go back to a previous commit (e.g., 1st commit)

```
# Copy commit ID of the first commit and run:  
git reset --hard <commit_id>
```

Check file contents after reset

```
cat calculator.sh
```

To share the code on GitHub, GitLab, or Bitbucket:

- # 1. Create a new repository on the platform (no template)
- # 2. Set repo name, description, visibility (private/public), add README if needed
- # 3. Push your local repo to remote:
 - # git remote add origin <remote_repo_url>
 - # git branch -M main
 - # git push -u origin main

What is branch:

- A branch is an independent line of development within a Git repository
- A branch is like creating a separate workspace for new features or changes so they don't affect the main application until ready.

Example:

- You build a simple calculator (main/master branch).
- You want to introduce version 2 with advanced features like % and mod.
- You create a new branch from master, work on these features, and test them.
- Once confident, you merge the branch back into master and delete it.

Reason for branching:

To ensure new changes don't break or affect existing functionality.

Real-world example:

- Uber initially only had cabs for customers.
- One day, they decided to add bikes.
- They created a **bikes** branch, developed and tested it, then merged it into the main branch.

git braching strategy

- **Primary Goal:** Ensure customers receive releases and new features on time.
- **Example Context (Kubernetes):**
 - Chosen as a reference because Kubernetes has ~3300 active contributors.
 - They follow a structured branching and release cycle.
 - New Kubernetes versions are released every 3 months, ensuring regular feature delivery and bug fixes.
- **Why it matters:** A well-defined branching strategy ensures smooth collaboration, predictable releases, and better code stability.

Git branching strategy

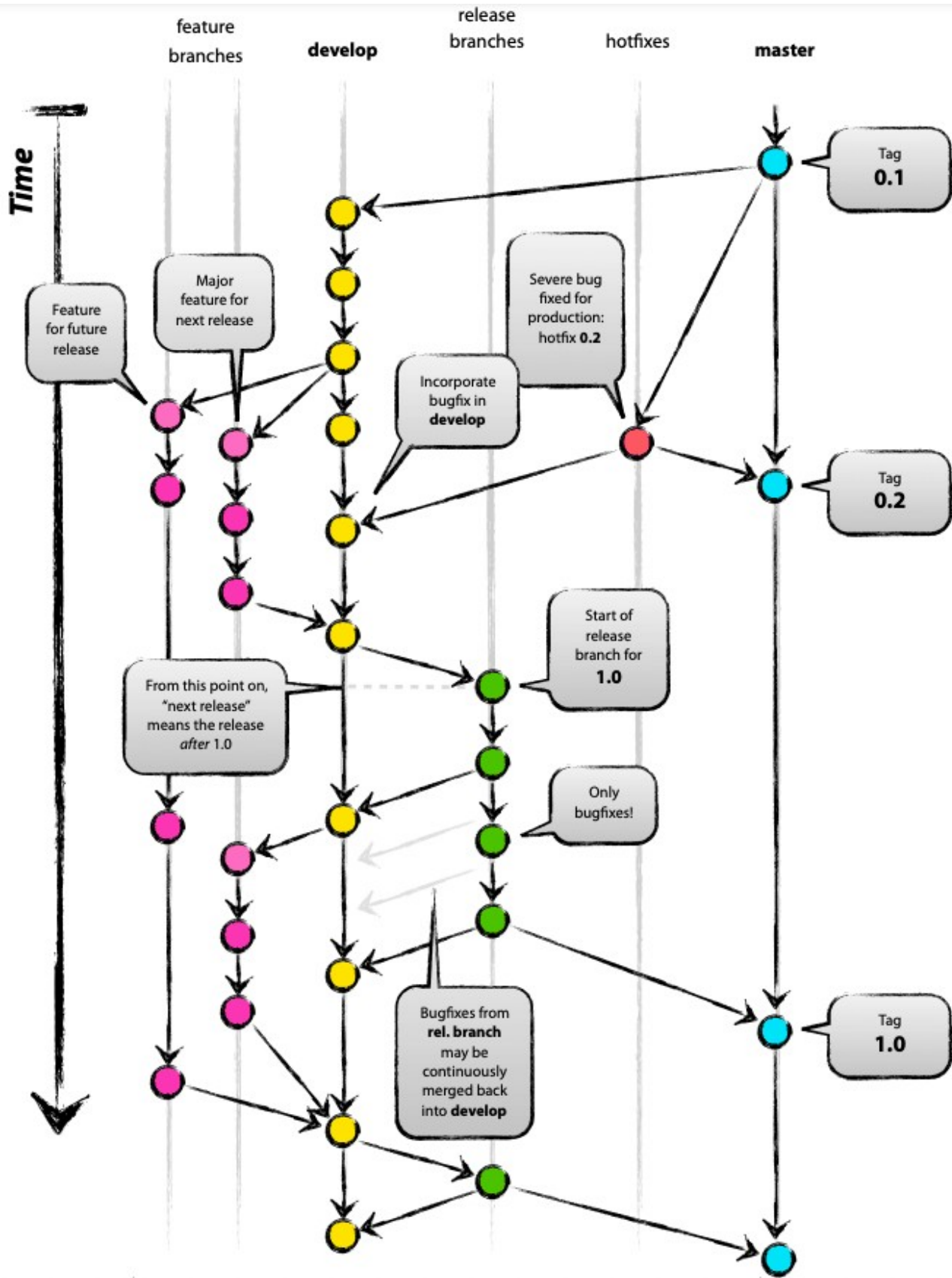
1.gitflow

2.githubflow

3.gitlab flow

3.trunk based flow

gitflow



Types of branches in git flow

1. master/main branch
2. feature branch
3. release branch
4. hotfix branch

Master and Feature branch

Branch Example – Calculator

- You have existing calculator code as of **15 Jan** — this is your **master/main branch** (main code base).
- You submit this version to the customer.
- Later, someone wants to add a new feature — so you create a **feature branch** (e.g., **feature-xyz**) from the **master branch**.
- You develop the feature in the **feature branch**, test it, and once confident, merge it back into the **master branch**.

In real projects:

- Multiple developers may work on different **feature branches** at the same time.
- Each feature branch is merged into the **master branch** after successful testing.

Release Branch

- When an application needs to be delivered to a customer, we usually **do not** build it directly from the **master branch**.
- Instead, we create a **release branch** from the **master branch**.
 - The **release branch** is used to:
 - Prepare the code for release
 - Perform final testing and bug fixes
 - Ensure stability before delivering to the customer

Reason:

- **Master branch** is meant for **active development** — new features and changes keep getting added.
- **Release branch** ensures that development can continue on **master** while the release code remains stable for deployment.

Workflow Example:

1. Development happens on the **master branch**.
2. When ready for release, create `release-v1.0` branch from **master**.
3. Perform final testing and bug fixes on `release-v1.0`.
4. Deliver build from `release-v1.0` to the customer.
5. Merge any final changes from `release-v1.0` back into **master** after release.

Hotfix branches

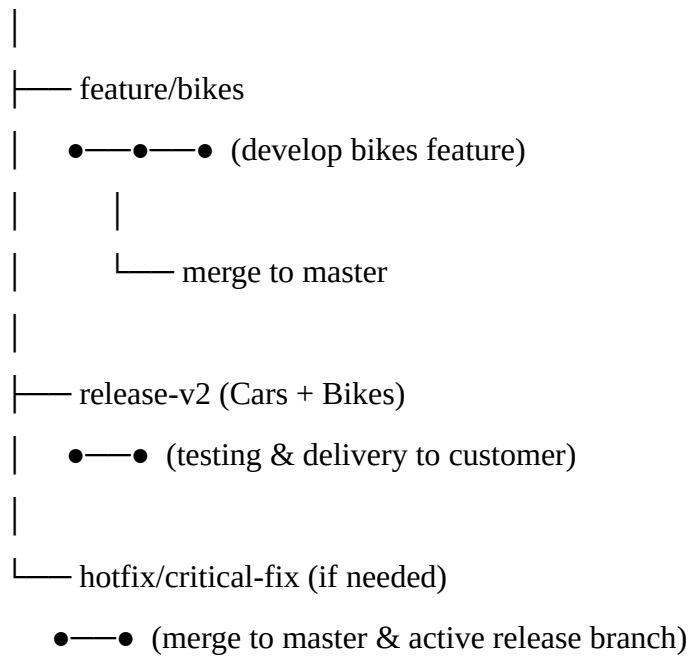
Hotfix Branch

- Used when there is a **critical issue in production** that must be fixed **immediately**.
- A **hotfix branch** is created directly from the **production branch** (often `master` or `main`).
- must be merged into both `master` and the release branch to preserve the fix in future releases.
- Purpose:
 - Quickly apply the fix without waiting for ongoing development or the next planned release.
 - Minimize downtime or impact to customers.
- **Timeline:** Usually very short-lived (1–2 days).
- **Workflow Example:**
 - Customer reports a severe bug in production.
 - Create a `hotfix-issue123` branch from `master`.
 - Implement the fix and test it quickly.
 - Deploy to production from the hotfix branch.
 - Merge the hotfix branch back into both **master** and **develop** (or relevant branches).
 - Delete the hotfix branch after deployment

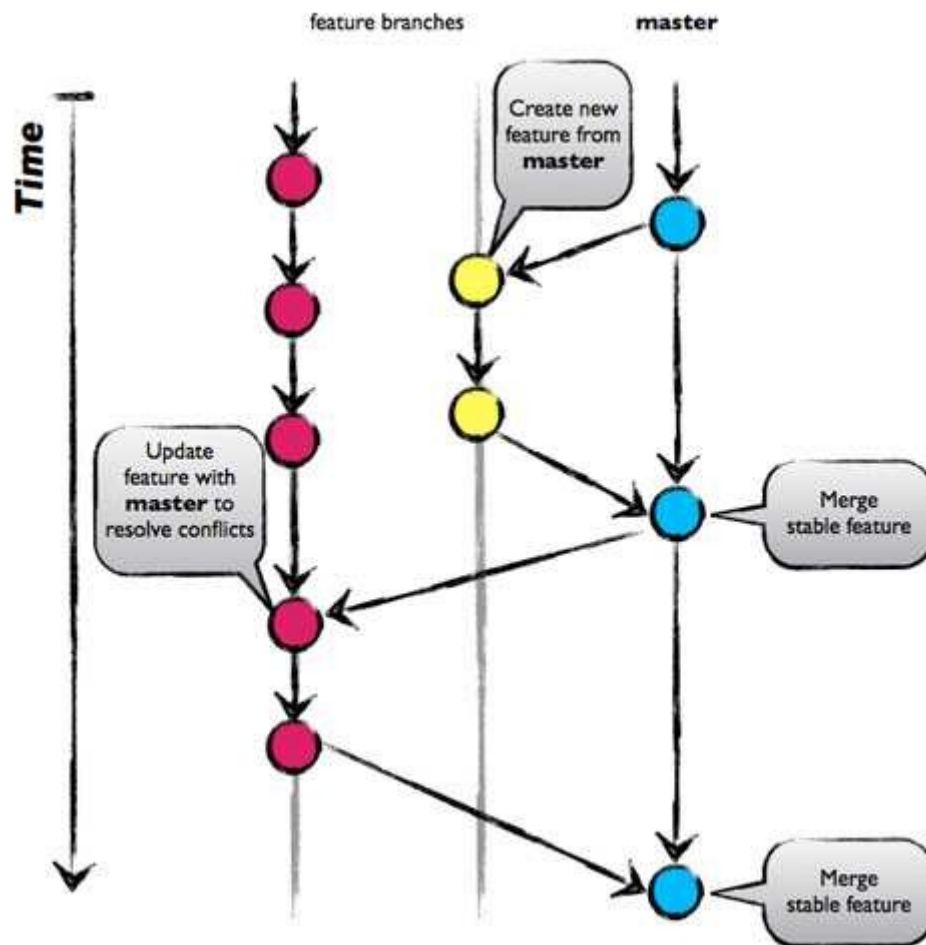
branching flow with all type of branches in gitflow

master

- (Start project: Cars feature ready)
|
|— release-v1 (Cars only)
| •—• (testing & delivery to customer)



GitHub Flow



Types of Branches in GitHub Flow

1. Main branch

- Always stable and deployable.
- Production is usually deployed directly from `main`.

2. Feature branches

- Short-lived (a few days).
- Created for new features, bug fixes, or experiments.
- Example: `feature/add-login`, `fix/button-style`.
- Merged into `main` via Pull Request (after review and testing).

Workflow Example – Calculator

1. On **Jan 15**, you deliver a stable calculator app from `main`.

2. A new request comes → *Add a scientific calculator mode.*
3. Create a new branch:

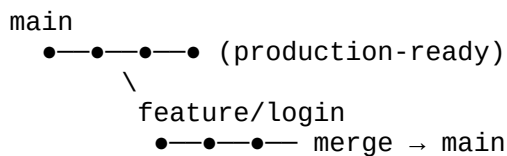
```
git checkout -b feature/scientific-mode
```
4. Develop and test the feature locally.
5. Push branch to GitHub:

```
git push origin feature/scientific-mode
```
6. Open a **Pull Request** on GitHub.
7. After code review + automated tests pass → merge into `main`.
8. Deploy immediately from `main`.

Key Differences from Git Flow

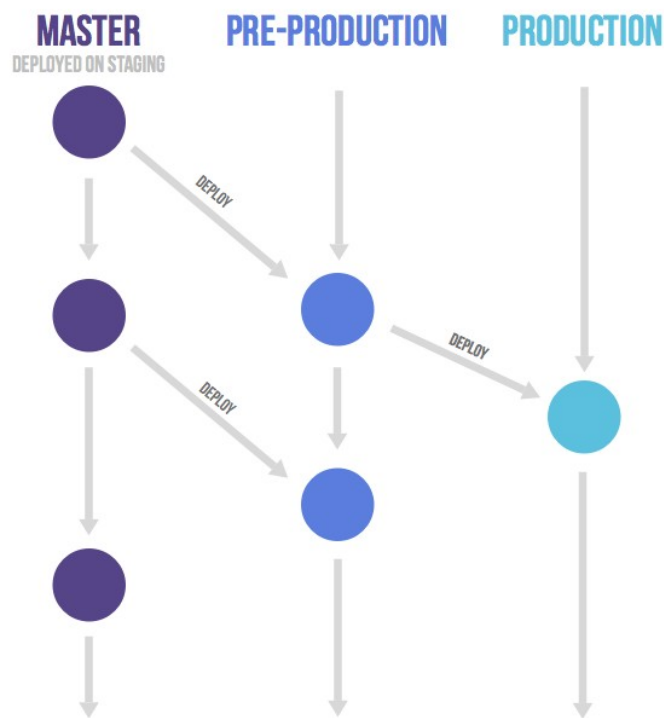
- No **develop** branch.
- No **release** or **hotfix** branches.
- Faster and simpler: just **main** + **feature branches**.
- Release is **continuous** (every merge to `main` is deployable).

Example Branching Flow (Graph)



Every feature branch is **short-lived** and merged back into `main` quickly.

Gitlab flow



GitLab Flow is a **branching strategy** that combines ideas from **GitHub Flow** (simplicity) and **Git Flow** (environment-based releases). It's widely used when teams need to align **development with deployment environments** (like dev, staging, production).

Key Points

GitLab Flow is **environment-driven**, not just feature-driven.
You usually have **main branches mapped to environments** like:

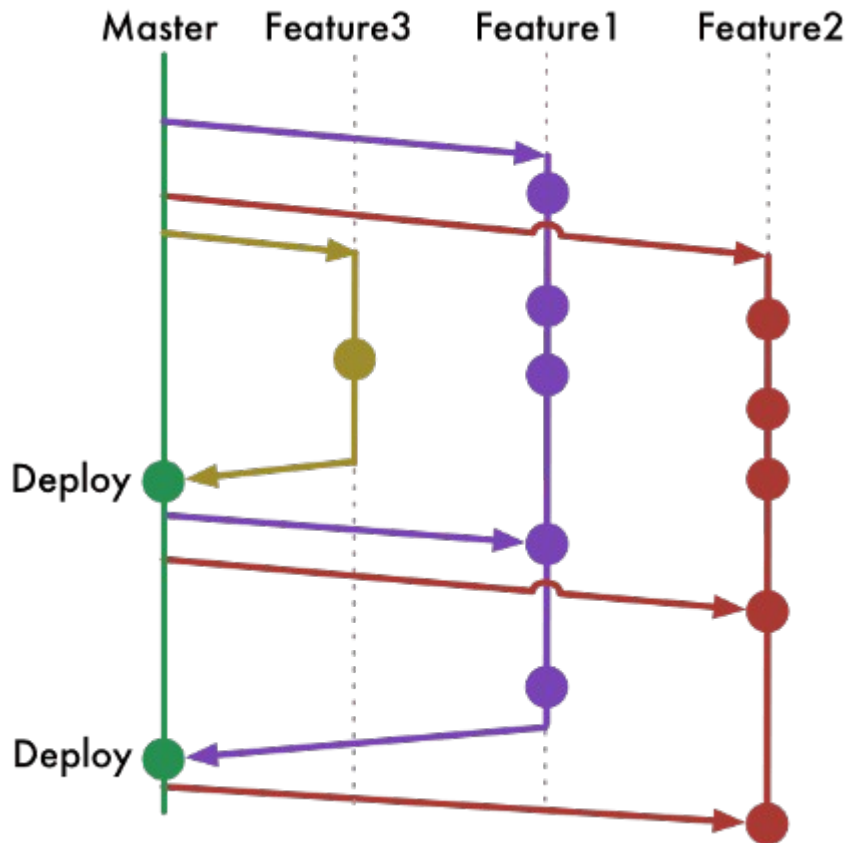
- `main` (production)
- `staging` (pre-production / QA)
- `development` (integration branch)

Developers create **feature branches** → merge into `development`.
Code is tested → merged into `staging` → finally into `main`.

When to Use GitLab Flow

- When your team has **multiple deployment environments** (Dev → QA → Prod).
- When you need a **clear promotion path** for features.
- When you want **continuous delivery with environment control**.

Trunk-based development



- In this strategy, all developers work on a **single main branch** (often called `main` or `master`).
- Instead of keeping long-lived feature branches, developers make **small, frequent commits** directly to the trunk.
- If a feature is big, they use **short-lived feature branches** (1–2 days max), merge quickly, and use **feature flags** to hide incomplete work.

Key Points to Mention in Interview

1. **One main branch (main/master)** → everyone commits/merges changes frequently.
2. **Short-lived feature branches** → avoid long-lived ones like in Git Flow.
3. **Feature Flags** → allow deploying incomplete features safely.
4. **Continuous Integration (CI)** → automated builds & tests run on every commit.

5. **Goal** → enable **faster releases**, reduce merge conflicts, and keep the code always deployable.

Pros

- Faster delivery (good for Agile/DevOps teams).
- Less merge conflicts.
- Codebase is always in a **release-ready state**.

Cons

- Needs **strong discipline** (small commits, good testing).
- Requires CI/CD and feature flag management.

git Q & A and commands for devops

Create project folder and script

```
mkdir gitdemo
cd gitdemo
vim calculator.sh    # Add: #!/bin/bash + # Addition function
```

Initialize git repository

```
git init
ls -la          # Verify .git folder created
```

Check status

```
git status      # Shows: Untracked file calculator.sh
```

Stage the file

```
git add calculator.sh
git status      # Shows: new file: calculator.sh
```

Edit file to add addition logic

```
vim calculator.sh    # Add: x=1+2
```

Check what changed before staging

```
git diff
```

Stage changes

```
git add calculator.sh
git status
```

Discard unstaged changes (if needed)

```
git checkout calculator.sh
```

Commit staged changes

```
git commit -m "first commit"
```

Verify working tree is clean

git status

View commit history

git log

commit 5b391956d8cf4ec0a578105a2ca60c828ce8e5b7 (HEAD -> master)

Author: JalajaDG <jalajagond@gmail.com>

Date: Wed Aug 13 15:21:59 2025 +0530

first commit

Push changes to remote repository

git push # Will show error if no remote repository is set

Fundamental workflow (common interview question)

Add changes -> Commit changes -> Push changes

git add -> git commit -m "" -> git push

Clone repository from GitHub

git clone <HTTPS-URL> # Replace <HTTPS-URL> with the repo link

git clone https://github.com/JalajaDG/samplerepo.git

It automatically creates a local repo, sets up the remote called `origin` pointing to the URL, and fetches all branches.

Enter the cloned repository

cd clonedrepo # Replace 'clonedrepo' with your repository folder name

cd **samplerepo**

Verify remote URL

git remote -v # Shows fetch and push URLs for origin

Example output:

origin https://github.com/JalajaDG/samplerepo.git (fetch)

origin https://github.com/JalajaDG/samplerepo.git (push)

Add remote repository (if remote not set or error occurs)

git remote add origin https://github.com/JalajaDG/samplerepo.git

Replace <HTTPS-URL> with your GitHub repo URL

This links your local repository to a remote repository

Verify remote is added

git remote -v # Should show origin fetch and push URLs

Push local commits to remote master/main branch

git push -u origin master # First push; after this, simple 'git push' works

git push -u origin master

Enumerating objects: 3, done.

Counting objects: 100% (3/3), done.

Writing objects: 100% (3/3), 251 bytes | 251.00 KiB/s, done.

Total 3 (delta 0), reused 0 (delta 0)

error: RPC failed; HTTP 403 curl 22 The requested URL returned error: 403

fatal: the remote end hung up unexpectedly

fatal: the remote end hung up unexpectedly

Everything up-to-date

HTTPS authentication issue

Generate a Personal Access Token (PAT) on GitHub

1. Go to **GitHub** → **Settings** → **Developer settings** → **Personal access tokens** → **Tokens (classic)** → **Generate new token** → **Generate new token (classic)**.
2. Give it a **name**, e.g., gitdemo-access.
3. Set **expiration** (e.g., 30 days or 90 days).
4. Select **repo** scope (this gives push/pull access to repositories).
5. Click **Generate token**.
6. **Copy the token immediately**. You won't be able to see it again.

token=

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

git remote set-url origin https://<username>:<PAT>@github.com/JalajaDG/samplerepo.git

git remote set-url origin

https://JalajaDG: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@github.com/JalajaDG/samplerepo.git

We have two methods to pull code from GitHub: HTTPS and SSH

1. Using HTTPS (requires password)

git clone <HTTPS-URL>

2. Using SSH (uses public/private key)

`ssh-keygen -t rsa` # Generate new SSH key

This creates `~/.ssh/id_rsa` and `~/.ssh/id_rsa.pub`

Copy the content of `id_rsa.pub`

Go to GitHub → Settings → SSH and GPG keys → New SSH key → Paste the key

git clone <SSH-URL>

Verify remote repository is linked

`git remote -v` # Shows fetch and push URLs for origin

Push local commits to remote repository (master/main branch)

`git push -u origin master` # First push; later simply 'git push' works

Complete SSH Git Setup Guide

1. Create Project Directory

```
mkdir sshgit
cd sshgit
```

2. Initialize Git Repository

```
git init
```

3. Generate SSH Key

```
ssh-keygen -t ed25519 -C "jalajagond@gmail.com"
```

• Press Enter 3 times (default location, no passphrase)

• Key saved to: `~/.ssh/id_ed25519` (private)

and `~/.ssh/id_ed25519.pub` (public)

4. Set Up SSH Agent

```
eval "$(ssh-agent -s)"  
ssh-add ~/.ssh/id_ed25519
```

5. Copy & Add Public Key to GitHub

```
cat ~/.ssh/id_ed25519.pub # Copy the entire output
```

1. Go to [GitHub SSH Keys](#)
2. Click "New SSH Key"
3. Paste your key and save

6. Test SSH Connection

```
ssh -T git@github.com
```

Should see: **Hi JalajaDG! You've successfully authenticated...**

Connecting to GitHub Repository

Option A: Existing Repository

```
git remote add origin git@github.com:JalajaDG/samplerepo.git
```

Option B: New Repository

1. First create repository on GitHub.com
2. Then run:

```
git remote add origin git@github.com:JalajaDG/new-repo-name.git
```

Verify remote:

```
git remote -v
```

First Commit & Push

1. Create Files

```
echo "# My Project" > README.md
```

2. Stage and Commit

```
git add README.md  
git commit -m "Initial commit"
```

3. Push to GitHub

For **master** branch:

```
git push -u origin master
```

For **main** branch:

```
git branch -M main # Rename branch if needed  
git push -u origin main
```

Key Notes

- SSH keys are stored in **~/.ssh/**
 - Always use **git@github.com:** URLs for SSH
 - First push requires **-u** flag to set upstream
 - Subsequent pushes can use just **git push**
-

Quick Reference Commands

```
# Check SSH connection  
ssh -T git@github.com
```

View remote URLs

`git remote -v`

Rename branch

`git branch -M new-name`

Fix wrong remote

`git remote set-url origin git@github.com:correct/repo.git`

This documents your entire working flow. You can save this as **GIT_SETUP_GUIDE.md** in your project for future reference!

git fetch vs git pull

- **git fetch:** Downloads changes from the remote repository into your local repository, but it does **not merge** them into your working branch. It just updates your remote tracking branches. You can later decide whether to merge or rebase those changes.
- **git pull:** It's basically `git fetch + git merge`. It fetches the latest changes from the remote and **directly merges them** into your current branch, so your working branch is updated immediately.

CLONE VS FORK

- **Clone** = “Download a repo into my computer.”
- **Fork** = “Make my own online copy of your repo, so I can change it and maybe send updates back.”

1. CLONE

- **Meaning:** Make a **local copy** of a repository (from GitHub or any Git server) to your machine.
- **Purpose:** You just want the code locally — to run, explore, or contribute.
- **Effect:** No GitHub link is created back to you unless you push somewhere.
- **Command:**

```
git clone https://github.com/user/repo.git
```
- **Example:** You find a public repo `user123/projectA` → you `git clone` it → now you can work on it offline.

2. FORK

- **Meaning:** Create **your own copy** of someone else’s repo **on your GitHub account**.
- **Purpose:** You want to make changes and maybe send them back via a pull request — without having write access to the original repo.
- **Effect:** GitHub creates a linked copy under your account. It remembers the "parent" repository.
- **Process:**
 1. On GitHub, click **Fork** on the repo page.
 2. This creates `yourname/projectA` on GitHub.
 3. You can then **clone your fork** locally:

```
git clone https://github.com/yourname/projectA.git
```

Key Difference

Action	Where Copy Exists	Need GitHub Account?	Linked to Original?
Clone	Only on your local machine	No	No
Fork	On your GitHub account (plus local after clone)	Yes	Yes, as a fork

see logs on branch

`git log branchname`

or

`git checkout branchname && git log`

`git cherry-pick comitid`

Merge

Example:

- git branch
* main
master
- git checkout -b multiply
Switched to a new branch 'multiply'
- git branch
main
master
* multiply
- vi file1.txt
//added line
multiplication=a*b
- git add file1.txt
- git commit -m "added multiplication to file1.txt"
[multiply 2bcd9df] added multiplication to file1.txt
1 file changed, 2 insertions(+)
- git log
commit 2bcd9dfd13a93ac1420de57b90b8a50cc57d9d0d (HEAD -> multiply)
Author: JalajaDG <jalajagond@gmail.com>
Date: Sun Aug 17 15:33:55 2025 +0530
added multiplication to file1.txt
- the changes of file1.txt is only on multiply branch ,not on main branch
 - git checkout main

- cat file1.txt
 - first line
 - second line
- git push origin multiply
- **now u want to merge those changes of multiply ibramches**

3 methods for merging :

- **Cherry-pick / Merge** → be on the branch that should **receive the changes**.
- **Rebase** → be on the branch that should **move** and be **rebased onto another**.

git cherry-pick

- Merge the commit of Multiply branch to main branch
- first get the commit id from multiply branch
 - git chekcout multiply
 - git log --oneline

2bcd9df (HEAD -> multiply, origin/multiply) added multiplication to file1.txt

ea0d8f9 (origin/main, main) added file2.txt

0def0ed modified file1.txt

823f35d (origin/master, master) added file1.txt

fc44583 Initial commit via SSH

- copy the commit id u want to move.**2bcd9df**

- **git chekcout main**

- git branch

* main

master

multiply

- git log --oneline **//before cherry-pick**

ea0d8f9 (HEAD -> main, origin/main) added file2.txt

0def0ed modified file1.txt

823f35d (origin/master, master) added file1.txt

fc44583 Initial commit via SSH

- **git cherry-pick 2bcd9df**

[main a7aa463] added multiplication to file1.txt

Date: Sun Aug 17 15:33:55 2025 +0530

1 file changed, 2 insertions(+)

- **git log --oneline //after cherry-pick**

a7aa463 (HEAD -> main) added multiplication to file1.txt

ea0d8f9 (origin/main) added file2.txt

0def0ed modified file1.txt

823f35d (origin/master, master) added file1.txt

fc44583 Initial commit via SSH

- the multiply branch also will have the commit

- git checkout multiply

- git log --oneline

2bcd9df (HEAD -> multiply, origin/multiply) added multiplication to file1.txt

ea0d8f9 (origin/main) added file2.txt

0def0ed modified file1.txt

823f35d (origin/master, master) added file1.txt

fc44583 Initial commit via SSH

Steps you did

1. **Checked out multiply** → got the commit ID 2bcd9df.
2. **Checked out main** → ready to apply the commit there.

3. **Cherry-picked 2bcd9df onto main** → Git created a *new commit* a7aa463 with the same changes.
4. After **cherry-pick**, both branches (**main** and **multiply**) now have *similar changes* to **file1.txt**, but they are represented by **different commit IDs**:
 - 2bcd9df in **multiply**
 - a7aa463 in **main**
5. This is why cherry-pick is different from merge: it **duplicates the commit change** into the other branch instead of unifying branch histories.
6. Cherry-pick is easy when there are 1-2 branches,,its not practically possible to manually copy each commit id if there rae 100's of branch

git Merge

- Create new branch merge-example
 - `git checkout -b Merge_example`
 - `git branch`
 - main
 - master
 - * merge_example
 - multiply
- lets add new functionality ie division in file1.txt
- `vi file1.txt`
 - //division =a/b
- `cat file1.txt`
 - first line
 - second line
 - multiplication=a*b
 - division= a/b
- `git add file1.txt`
- `git commit -m "added division functionality to file1.txt"`

- `git log --oneline`

ff02181 (HEAD -> merge_example) added division functionality to file1.txt

2bcd9df (origin/multiply, multiply) added multiplication to file1.txt

ea0d8f9 (origin/main) added file2.txt

0def0ed modified file1.txt

823f35d (origin/master, master) added file1.txt

fc44583 Initial commit via SSH

- edit last commit msg
 - `git commit --amend`
 - Merge Demo : added division functionality to file1.txt
 - `ctrl +o`
 - `enter`
 - `ctrl+x`

`git log --oneline`

- **git checkout main**
- **git merge merge_example**
- **u have a merge conflict**

Auto-merging file1.txt

CONFLICT (content): Merge conflict in file1.txt

Automatic merge failed; fix conflicts and then commit the result.

- Nano file1.txt

first line

second line

multiplication=a*b

<<<<<<< HEAD

=====

division= a/b

>>>>>>> merge_example

- HEAD = **your current branch** (main)
- merge_example = the branch you were merging in

So:

- multiplication=a*b came from **main**
- division=a/b came from **merge_example**

Use **git show :1:filename / :2: / :3:**

During a conflict, Git actually saves **three versions** of the file:

- :1:file1.txt → common ancestor
- :2:file1.txt → your branch (main / HEAD)
- :3:file1.txt → merging branch (merge_example)

So you can see exactly what each side contributed:

```
git show :2:file1.txt    # main branch version
git show :3:file1.txt    # merge_example branch version
```

```
git show :2:file1.txt
```

first line

second line

multiplication=a*b

```
git show :3:file1.txt
```

first line

second line

multiplication=a*b

division= a/b

Since you want **both features** (multiplication and division), just edit the file manually to remove the markers and keep both lines:

- **nano file1.txt**

```
first line
second line
```

```
multiplication=a*b
division= a/b
```

- `git add file1.txt`
- `git commit`
[main 1c14e61] Merge branch 'merge_example' into main

- **git log --oneline**

```
1c14e61 (HEAD -> main) Merge branch 'merge_example' into main
e83f5f6 (merge_example) Merge Demo : added division functionality to file1.txt
a7aa463 added multiplication to file1.txt
2bcd9df (origin/multiply, multiply) added multiplication to file1.txt
ea0d8f9 (origin/main) added file2.txt
0def0ed modified file1.txt
823f35d (origin/master, master) added file1.txt
fc44583 Initial commit via SSH
```

Why conflict happened

Normally, this should be an **automatic merge** (Git can just add the new line).

But — in your history:

- On main, `multiplication=a*b` came from a **cherry-pick** (a7aa463).
- On merge_example, `multiplication=a*b` came from a **different commit** (the merge demo).

To Git, the two lines look the same *textually*, but they were introduced by **different commits**. This makes Git extra cautious → it flags a conflict rather than assuming.

Check the commit history of each branch

git log --oneline -p main -- file1.txt

git rebase

- Git checkout main
- git branch
 - * main
 - master
 - merge_example
 - multiply
- git checkout -b rebase_example
- git branch
 - main
 - master
 - merge_example
 - multiply
 - * rebase_example
- vi file1.txt
 - //here there is no division functionality bcz it was added insdie merge_example branch not in main branch
 - //add mod functionality
- cat file1.txt
 - first line
 - second line
 - multiplication=a*b
 - mod= a%b
- git diff
- git add file1.txt
- git commit -m "rebase demo : added mod functionality"
- git log --oneline

41719e2 (HEAD -> rebase_example) rebase demo : added mod functionality

- Switch to the branch you want to rebase
- **git checkout rebase_example**
- cat file1.txt

first line

second line

multiplication=a*b

mod= a%b

- **lest see file in main branch also (better to look, to check for merge conflicts)**
 - git checkout main
 - **cat file1.txt**

first line

second line

multiplication=a*b

division= a/b

- **git checkout rebase_example**
- **git rebase main**

First, rewinding head to replay your work on top of it...

Applying: rebase demo : added mod functionality

Using index info to reconstruct a base tree...

M file1.txt

Falling back to patching base and 3-way merge...

Auto-merging file1.txt

CONFLICT (content): Merge conflict in file1.txt

error: Failed to merge in the changes.

Patch failed at 0001 rebase demo : added mod functionality

hint: Use 'git am --show-current-patch' to see the failed patch

Resolve all conflicts manually, mark them as resolved with

"git add/rm <conflicted_files>", then run "git rebase --continue".

You can instead skip this commit: run "git rebase --skip".

To abort and get back to the state before "git rebase", run "git rebase --abort".

ee213082@lpz-ee213082:~/Desktop/sshgit\$

- `cat file1.txt`

first line
second line

multiplication=a*b

```
<<<<<< HEAD
division= a/b
=====
mod= a%b
>>>>>> rebase demo : added mod functionality
```

- **`vi file.txt`**

first line
second line

multiplication=a*b

division= a/b
mod= a%b

- **`git add file1.txt`**
- **`git rebase --continue`**
Applying: rebase demo : added mod functionality

exmpale

STEP 0: Start from main branch
`git checkout main`

STEP 1: Create branch1 and add commits

`git checkout -b branch1` # create and switch to branch1

Commit 1

`touch file3.txt`

`echo "Branch1 - line 1" > file3.txt`

`git add file3.txt`

`git commit -m "branch1: first commit in file3.txt"`

Commit 2

`echo "Branch1 - line 2" >> file3.txt`

`git add file3.txt`

`git commit -m "branch1: second commit in file3.txt"`

Visualization:

```
# main
# |
# └─ branch1
#   └─ branch1: first commit
#   └─ branch1: second commit
```

```
# -----
```

STEP 2: Create branch2 from branch1 and add commits

```
# -----
```

```
git checkout -b branch2    # create and switch to branch2
```

```
# Commit 1
```

```
echo "Branch2 - line 1" >> file3.txt
```

```
git add file3.txt
```

```
git commit -m "branch2: first commit in file3.txt"
```

```
# Commit 2
```

```
echo "Branch2 - line 2" >> file3.txt
```

```
git add file3.txt
```

```
git commit -m "branch2: second commit in file3.txt"
```

```
# Visualization:
```

```
# main
# |
# └─ branch1
#   └─ branch1: first commit
#   └─ branch1: second commit
#     \
#      branch2
#       └─ branch2: first commit
#       └─ branch2: second commit
```

```
# -----
```

STEP 3: Add another commit to branch1 (to create divergence)

```
# -----
```

```
git checkout branch1
```

```
echo "Branch1 - line 3" >> file3.txt
```

```
git add file3.txt
```

```
git commit -m "branch1: third commit in file3.txt"
```

```
# Visualization:
```

```
# main
# |
# └─ branch1
#   └─ branch1: first commit
#   └─ branch1: second commit
#   └─ branch1: third commit
#     \
#      branch2
#       └─ branch2: first commit
#       └─ branch2: second commit
```

```
# -----
# STEP 4: Rebase branch2 onto branch1
# -----
git checkout branch2
git rebase branch1

# -----
# STEP 5: Resolve conflicts (if any)
# -----
# If you get conflicts in file3.txt:
# 1. Open file3.txt in your editor
# 2. Remove conflict markers <<<<<<<, =====, >>>>>>>
# 3. Merge the changes manually
# 4. Save the file
# 5. Continue the rebase
git add file3.txt
git rebase --continue

# Repeat until all conflicts are resolved

# Visualization during rebase (linearized history):
# branch1
# |— branch1: first commit
# |— branch1: second commit
# |— branch1: third commit
# |— branch2 (rebased)
# |   |— branch2: first commit
# |   |— branch2: second commit

# -----
# STEP 6: Verify rebase history
# -----
git log --oneline --graph --all

# Expected output (linear history):
# * branch2: second commit
# * branch2: first commit
# * branch1: third commit
# * branch1: second commit
# * branch1: first commit
# * main commit before branches
```

Merge vs Rebase in Git (commit history)

1. Merge

- When you do `git merge`, Git creates a **new merge commit** (like your `1c14e61 Merge branch 'merge_example' into main`).
- That's why you see *a new commit* in `git log`.

2. Rebase

- When you do `git rebase`, Git **does not create a merge commit**.
 - Instead, it **re-applies your commits on top of the target branch's commits**, rewriting history.
 - This means the commit IDs (SHA) of your rebased commits **change**, because Git creates *new versions* of those commits.
- **Git Merge**
Combines changes from one branch into another by creating a new merge commit. This keeps the branch history intact.
 - **Git Rebase**
Moves or reapplies commits from one branch **on top of another branch**. This rewrites history (no extra merge commit) and gives a **linear commit history**.
-

Differences Between Merge and Rebase

Feature	Merge	Rebase
Commit History	Keeps full branch history (non-linear, with merge commits).	Creates a linear history (no merge commits).
Extra Commit	Yes, a new merge commit is added.	No extra commit; existing commits are reapplied.
Clarity	Shows when and where branches diverged and merged.	Cleaner, linear history without branching info.
Use Case	Good when you want to preserve the full history of collaboration.	Good when you want a clean, linear project history.
Safe for Shared Branches?	Safe (does not rewrite history).	Risky for shared branches (rewrites history).

use merge; when u want to preserve history and context
 use rebase; when u want clean and linear history

Example Timeline

Step 1: Start

```
main:    A -- B
          ↑
        (common base)
feature:  \
          C -- D
```

- A and B are commits on `main`.
 - You created `feature` at commit B, then made two new commits C and D.
-

Case 1: Merge

You want to bring `main` and `feature` together.

Command:

```
git checkout main
git merge feature
```

Result:

```
main:    A -- B ----- E
          \           /
feature:  C - D
```

- Git makes a **new commit E** (merge commit).
 - Both histories are preserved.
 - It looks like: *"main + feature joined at E"*.
-

Case 2: Rebase

Instead of creating a merge commit, you say:

"Pretend I made my changes after the latest `main` commit."

Command:

```
git checkout feature
git rebase main
```

Result:

```
main:    A -- B
          \
feature:  C' -- D'
```

- Commits C and D are **rebuilt** on top of B.
- Looks like you started your work **after the latest main** commit.

- No merge commit, cleaner history.

Difference

- **Merge:** "Combine our histories, show both paths."
Keeps record of how branches diverged.
History looks messier.
- **Rebase:** "Pretend I never diverged, replay my work on top."
Clean, linear history.
Rewrites commit IDs ($C \rightarrow C'$, $D \rightarrow D'$).

Feature / Aspect	Git Merge	Git Rebase
Purpose	Combines two branches while keeping history intact	Moves a branch's commits on top of another branch (linearizes history)
History	Non-linear (creates a merge commit)	Linear (no merge commits; rewrites commit history)
Commit Creation	Creates a new "merge commit"	Does not create a merge commit; reapplies commits
Workflow	Good for preserving context of feature work	Good for clean, linear project history before merging into main
Conflict Handling	Conflicts resolved once during merge	Conflicts resolved at each commit being rebased
Command Example	<code>git checkout main</code> <code>git merge branch2</code>	<code>git checkout branch2</code> <code>git rebase main</code>
History After Operation	Shows branching structure (non-linear graph)	Shows straight line history (linear graph)
Undo	Can be undone easily using <code>git reset</code> or <code>git revert</code>	More complicated if pushed to shared repo, may require <code>git reflog</code>
Use Case	When you want to combine changes and preserve the context of branch history	When you want to integrate changes cleanly without extra merge commits, usually before merging into main branch
Visual Representation	<pre> main ——— merge commit branch2 * commit2 * commit1 </pre>	<pre> main —*—*—* branch2 rebased commits on top </pre>

How can you rollback the commit in git

1. `git reset --soft HEAD~1`

Moves HEAD back by 1 commit..Keeps changes in **staging area**.

Example:

C0 --- C1 --- C2 (HEAD, master)

Run:

```
git reset --soft HEAD~1
```

Now:

- Branch points to C1 instead of C2.
- All changes from C2 are **staged**, ready to be recommitted.
- Working directory still has those changes.

Use when: You want to "undo commit but keep changes staged".

```
git reset --soft HEAD~1
```

```
# edit files
```

```
git commit -m "better message"
```

2. `git reset --mixed HEAD~1` (default if you just do `git reset HEAD~1`)

Moves HEAD back by 1 commit.

- Removes changes from **staging area**.
- Keeps changes in **working directory**.

Example:

```
git reset HEAD~1
```

Now:

- Branch points to C1.
- Changes from C2 go to **working directory** (unstaged).
- Nothing is lost, but you need `git add` again to stage.

Use when: You want to "undo commit and unstage changes but keep files modified".

```
git reset HEAD~1
```

```
# edit files again
```

```
git add .
```

```
git commit -m "clean commit"
```

3. **git reset --hard HEAD~1**

Moves HEAD back by 1 commit.

- Discards changes in staging area.
- Discards changes in working directory.

Example:

```
git reset --hard HEAD~1
```

Now:

- Branch points to C1.
- Changes from C2 are **gone completely** (unless in reflog).

Dangerous: Work will be lost (unless you recover with `git reflog`).

Use when: You want to "undo commit and throw away all changes".

```
git reset --hard HEAD~1
```

```
# commit totally erased
```

Command	Commit Removed?	Staging Area	Working Directory
--soft	Yes	Keep changes staged	Keep changes
--mixed (default)	Yes	Unstage changes	Keep changes
--hard	Yes	Discard changes	Discard changes

git revert <commit-hash>

- `git revert <commit-hash>` **does not delete** the commit from history.
- Instead, it creates a **new commit** that introduces the **inverse changes** of the specified commit.
- This way, the project history remains intact (important in shared branches like `main` or `master`).

Example with Multiple Reverts

You can also revert multiple commits at once:

```
git revert <hash1> <hash2>
```

Or a range:

```
git revert HEAD~2..HEAD
```

(undoes the last 2 commits).

Why Use `git revert` Instead of `git reset`?

- **Safe in shared repos:** Doesn't rewrite history, so teammates don't face conflicts.
- **Transparent:** You can see when and why something was undone.
- Creates an extra commit (history gets longer).

Git stash

It's a command used to temporarily save (stash) your changes without committing them.

Temporarily saves (stashes) your uncommitted changes and cleans your working directory. Changes are stored in a stack (like push/pop).

This is useful when:

- You are working on something, but need to quickly switch branches.
- You want to pull new changes from remote, but you have uncommitted changes.

Exmample:

You're on feature/cart, and you've edited login.c but haven't staged or committed yet. Now you're told to fix a bug in main.

That's why Git won't let you `git checkout main` — because switching branches would overwrite your uncommitted changes.

Example Scene

Imagine you are working on a project in the `feature-login` branch.

1. You make some changes

```
git checkout feature-login
vim login.c # make some changes
```

Check status:

```
git status
```

Output:

```
Changes not staged for commit:
  modified: login.c
```

2. Suddenly, you are asked to fix a bug in main branch

But you cannot switch branches because you have uncommitted changes.

If you try:

```
git checkout main
```

You'll see:

```
error: Your local changes would be overwritten by checkout.
```

3. Use `git stash`

To save changes temporarily:

`git stash`

Now check status:

`git status`

Output:

`working tree clean`

Your changes are safe in the stash, and your workspace is clean.
Now you can switch to `main` branch and fix the bug.

4. View stashed changes

`git stash list`

Output:

`stash@{0}: WIP on feature-login: 1234abc some commit message`

5. Apply stashed changes

Once you finish bugfix and return to `feature-login`, you can restore changes:

`git checkout feature-login`
`git stash apply`

Now your changes in `login.c` are back.

6. If you want to remove stash after applying

`git stash drop stash@{0}`

Just **delete** the stash entry without applying

OR (apply + remove in one step):

`git stash pop`

- Restores the latest stashed changes **and removes them from the stash stack.**
- Think of it like: *"Take it back and delete from stash."*
- *After `pop`, stash list is empty if only one was there.*

Summary:

- `git stash` → save changes temporarily
- `git stash list` → see all stashes
- `git stash apply` → get them back
- `git stash pop` → apply + delete stash
- `git stash drop` → delete specific stash

Basic Git Questions

1. What is Git?

- **Git is a *Distributed Version Control System (DVCS)*** used to track changes in source code during software development.
- It allows multiple developers to work simultaneously,
- s
- **distributed version control system (DVCS)**, which means every developer has a full copy of the repository, including its complete history, on their local machine

2. How is Git different from other version control systems like SVN?

- SVN is *centralized* → there is one central repository, and all developers must connect to it.
- Git is *distributed* → every developer has a full copy of the repository (including history).

Feature	Git (Distributed)	SVN (Centralized)
Architecture	Distributed – every developer has a full local copy of the repo	Centralized – single central repo on server
Availability	Work offline (commits, diffs, logs possible locally)	Needs internet/server access for most operations
Speed	Faster – operations are local	Slower – server round-trip needed
Fault Tolerance	Safe – even if server crashes, devs still have full history	Risky – if server crashes, repo access lost
Branching & Merging	Lightweight, very fast	Heavy, slower
History	Complete repo history on every machine	History stored only on central server
Setup	Initial clone is larger (downloads full repo)	Checkout is smaller, but needs server for history
Security	Each copy is independent; less chance of total data loss	Single point of failure – central server

•

3. What is the difference between Git and GitHub?

Aspect	Git	GitHub
Definition	A Distributed Version Control System (DVCS) used to track changes in source code during s/w development. allow multiple developers to work simultaneously, supports branching, merging & maintain repo+history locally	A cloud-based hosting platform for Git repositories.

Aspect	Git	GitHub
Nature	A tool/software installed locally on your system.	A service/platform hosted on the internet.
Usage	Manages code history, branching, merging, and local repositories.	Provides remote hosting, collaboration, issue tracking, pull requests, and CI/CD integrations.
Where it runs	Runs locally on developer machines.	Runs on the web (GitHub servers).
Access	Does not require internet to use (except for remote sync).	Requires internet to access repositories hosted on GitHub.
Collaboration	Collaboration possible via local sharing or pushing to a remote.	Provides built-in collaboration features (pull requests, code reviews, discussions).
Ownership	Free and open-source software developed by Linus Torvalds.	Owned by Microsoft (acquired in 2018).

4. How do you initialize a Git repository?

- `git init` - this creates new empty git repository in your current directory
- It adds a hidden **.git/** folder where Git stores all its metadata, commits, and version history.
- After running it, you can start tracking files with `git add` and commit with `git commit`.

5. How do you clone a repository?

By using the `git clone` command.

- You can clone using **HTTPS** or **SSH** URLs.
- Example:
 - `git clone https://github.com/user/repo.git` (HTTPS)
 - `git clone git@github.com:user/repo.git` (SSH)

6. What is the difference between `git clone` and `git init`?

- `git init` → Initializes a **new, empty Git repository** in the current directory.
- `git clone <repo>` → Creates a **local copy of an existing remote repository** along with its history and sets up the remote origin.

7. What is a commit in Git?

- **commit** in Git is a **snapshot of the changes** in the repository at a specific point in time.
- Each commit has a **unique SHA-1 hash** that identifies it in the history.
- Commits form a **timeline/history** of the project, allowing you to revert, branch, or compare changes.
- the last saved version of the file in Git history.
-

8. How do you check the current status of a repository?

Git status

- Shows which branch you are on.
- Lists **staged** changes ready to commit.
- Lists **unstaged** changes in your working directory.
- Shows **untracked** files (files not yet added to Git).

Action	Git Status Result	git restore / Undo Command
New file	Untracked	<code>git clean -f <file></code> → deletes untracked file
<code>git add</code>	Staged (Changes to be committed)	<code>git restore --staged <file></code> → unstage the file
<code>git commit</code>	Committed, working tree clean	N/A (cannot undo commit with restore; use <code>git reset --soft HEAD~1</code> to undo commit)
Modify committed file	Unstaged changes	<code>git restore <file></code> → discard changes and revert to last commit
<code>git add</code> after modification	Staged again	<code>git restore --staged <file></code> → unstage staged changes
Discard all changes	Working tree clean	<code>git restore <file></code> or <code>git restore --staged <file></code>

9. What does `git diff` do?

- `git diff` shows the differences between files in different Git areas. By default, it shows **unstaged changes** (working directory vs staging area).
 - Shows difference between files in working directory and staging area
- `git diff --staged` shows **staged changes** (staging area vs last commit).

- `git diff HEAD` shows **all changes since the last commit** (working directory vs last commit).

10. What is the difference between `git pull` and `git fetch`?

- **`git fetch`**: Downloads changes from the remote repository into your **local repository** (updates remote-tracking branches) but does **not modify your working directory or current branch**.
 - **`git pull`**: `git fetch + git merge`.
 - **`git pull`**: Downloads changes and **merges (or rebases)** them into your current branch, updating your working directory.
-

Branching & Merging

11. What is a branch in Git?

- **A branch in Git** is a separate line of development in a repository, allowing you to work on features, fixes, or experiments independently without affecting the main codebase.

12. How do you create a new branch?

`git branch branchname` : Create a branch without switching

`git checkout -b branchname` : creates and switch to new branch

`git switch -c branchname` : Recommended in newer Git versions.

13. How do you switch between branches?

`git checkout branchName`

`git switch branchname`

14. How do you merge branches in Git?

- To merge branches in Git, first switch to the branch you want to merge **into** (e.g., `main`) and run:
 - `git merge featureBranch`
- To rebase a branch onto another, switch to the branch you want to **rebase** (e.g., `featureBranch`) and run:
 - `git rebase main`
- Optional short addition if asked:
 - **Merge** preserves branch history.

- **Rebase** rewrites commits to create a linear history.

15. What are fast-forward merges vs. three-way merges?

- **Fast-forward merge:** Happens when the target branch has no new commits; Git just moves the branch pointer forward—no merge commit is created.

main: A---B

feature: A---B---C---D

Merging feature into main → main pointer moves to D.

- Here, **main** is the **target branch** because you are merging **feature** into it.
- **feature** is the **source branch** (the branch whose changes you want to bring in).
- **Three-way merge:** Happens when branches have diverged; Git combines changes from both branches and creates a new merge commit.

main: A---B---C

feature: B---D

Merging feature into main → Git creates a merge commit combining C and D.

16. How do you resolve a merge conflict?

- Open the conflicting file(s) marked by Git.
- Manually edit to choose which changes to keep.
- Save the file(s) and stage the resolution with `git add`.
- Complete the merge with `git commit` (if not auto-committed).

17. What is the difference between `git merge` and `git rebase`?

- **Git merge:** Combines changes from one branch into another and creates a **merge commit**, preserving the branch history.

- **Git Merge Example**

main: A---B

feature: C---D

- On **main**, run:

```
git checkout main
git merge feature
```

- Result:

```
main:    A---B---M
feature:  C---D
```

- M = merge commit combining main and feature. History is preserved.

- git rebase; move or reapply the commits of 1 branch on top of another branch, no merge commit, keeps clean, linear history

- **Git Rebase Example**

```
main:    A---B
feature:  C---D
```

- On feature, run:

```
git checkout feature
git rebase main
```

- Result:

```
main:    A---B
feature:  C'---D'
```

- Commits C and D are **reapplied on top of main**. No merge commit; history is linear.
- **rebase example**, C' and D' are **new commits**.

18. When would you use **git rebase** instead of **git merge**?

- Use **git rebase** when you want a clean, linear history without merge commits.
- Use **git merge** when you want to preserve the complete branch history, including all merges.
-

19. How do you delete a branch locally and remotely?

Action	Command	Notes
Delete local branch (safe)	<code>git branch -d branchname</code>	Only deletes if branch is fully merged
Delete local branch (force)	<code>git branch -D branchname</code>	Deletes even if branch is not merged

Action	Command	Notes
Delete remote branch	<code>git push origin --delete branchname</code>	Removes branch from remote repository
Delete remote branch (old)	<code>git push origin :branchname</code>	Alternative older syntax

20.

21.What does HEAD mean in Git?

- **HEAD** is a pointer that represents the **current commit your working directory is based on**—usually the **latest commit on the current branch**.
- When you switch branches, **HEAD moves** to point to the tip of that branch.
- You can also make **detached HEAD** if you checkout a specific commit instead of a branch.

Staging & Commit Management

21.What is the staging area in Git?

After `git add`, files enter the staging area (index), where changes are prepared and ready for the next commit.

22.How do you add changes to the staging area?

“`git add` stages changes, preparing them for the next commit.”

23.What is the difference between `git add .` and `git add -A`?

Command	What it does
<code>git add .</code>	Stages new and modified files in the current directory and its subdirectories, ignores deleted files .
<code>git add -A</code>	Stages all changes , including new, modified, and deleted files , across the entire repository.

24.How do you undo a staged file?

Git restore --staged filename.unstage a file without losing its changes;

25.How do you modify the last commit message?

- `Git commit -amend`
- this opens the editor

- After saving, the last commit is replaced with a **new commit** (new commit ID).
- Note: Avoid amending commits that have already been **pushed** to a shared remote, as it rewrites history.

26. How do you undo the last commit without losing changes?

git reset --soft HEAD~1

- `--soft` keeps all your changes in the staging area.
- `git reset --mixed HEAD~1` → keeps changes in the **working directory**, unstaged.
- `git reset --hard HEAD~1` → discards both commit and changes (use with caution).
- `restore` → undo **uncommitted changes** or unstage files.
- `reset` → undo commits locally and optionally modify staged/working changes (rewrites history).
- `revert` → undo a commit by **creating a new commit**, safe for shared repos.

27. How do you completely remove the last commit?

“`git reset --hard HEAD~1` deletes the last commit and its changes.”

28. What is the difference between `git reset`, `git checkout`, and `git revert`?

Command	What it Does	Effect on History	Safe for Shared Repo?	Example
<code>git reset</code>	Moves branch pointer to a previous commit; can unstage or discard changes	Rewrites history	Not safe if pushed	<code>git reset --soft HEAD~1</code>
<code>git checkout</code>	Switches branches or restores files in working directory	Does not change history	Safe	<code>git checkout HEAD -- file.txt</code> (restores file to last commit)
<code>git revert</code>	Creates a new commit that undoes a previous commit	Preserves history	Safe	<code>git revert abc123</code> (undoes commit abc123 by making a new commit)

29.What are soft, mixed, and hard resets in Git?

Reset Type	What It Does	Where Changes Go
--soft	Moves branch pointer to a previous commit	Changes stay staged (index)
--mixed	Moves branch pointer to a previous commit (default)	Changes stay in working directory, unstaged
--hard	Moves branch pointer and discard all changes	Changes removed from working directory and staging

30.How do you squash multiple commits into one?

- Git rebase -i HEAD~N
 - i=interactive
 - N=no of commits to squash
 - Keep **pick** for the **first commit**.
 - Change **pick** → **squash** (or **s**) for the **subsequent commits** you want to combine.
 - Save and close the editor.
 - Git will let you **edit the commit message** for the squashed commit.
 - See git log
 -
-

Remote Repositories

31.How do you add a remote repository?

- `git remote add origin <URL>` to link a local repo to a remote repository

32.How do you list remote repositories?

- `git remote -v`
- `-v` shows both **fetch** and **push** URLs.
- “`git remote -v` lists all remotes with their fetch and push URLs.”

33.What is the difference between **origin** and **upstream**?

Name	Meaning / Use
origin	Default name for the remote repository you cloned from or primarily push to .
upstream	Typically refers to the original repository from which your fork was created; used to pull updates from the main source.

34.How do you push changes to a remote branch?

- Git push origin branchname
 - origin → name of the remote (default for the repository you cloned).
 - branchname → the branch you want to push.
- If pushing a new branch for the first time, you may add -u to set upstream:
 - git push -u origin branchname **will push a new local branch** to the remote repository if it doesn't already exist there.

35.How do you push a new branch to the remote repository?

- git push -u origin branchname will push a new local branch to the remote repository if it doesn't already exist there.

36.How do you pull changes from a remote repository?

- To fetch and merge changes from a remote repository into your current branch:git pull <remote> <branch>
- git pull origin main
- origin → remote repository name.
- main → branch you want to pull from.
- git pull is essentially git fetch + git merge.

37.What is the difference between git fetch and git pull -rebase?

•

Command	What it Does
git fetch	Downloads all changes from the remote repository to your local repo , but does not merge them into your working directory or current branch.
git pull --rebase	Downloads changes from the remote and rebases your local commits on top of the remote changes. This keeps a linear history instead of creating a merge commit.

38.How do you set up tracking for a remote branch?

1.When pushing a new branch:

```
git push -u origin <branchname>
```

- The `-u` (or `--set-upstream`) flag **links your local branch to the remote branch**.
- After this, you can just run `git pull` or `git push` without specifying the branch.

2. If the branch already exists on remote:

```
git checkout --track origin/<branchname/feature1>
```

- Git creates a **local branch** called `feature1` that **tracks** `origin/feature1`.
- Creates a local branch that **tracks the remote branch** automatically.
- **Use `--track`** when a remote branch exists but you don't have it locally yet.
- It links your local branch to the remote so future `pull/push` commands are automatic.
-

39.How do you remove a remote repository?

```
git remote remove <remote-name>
```

```
git remote remove origin
```

- Removes the remote reference from your local repository.
- Does **not delete the remote repository itself**; only disconnects your local repo from it.

40.How do you rename a remote repository?

```
git remote rename <old-name> <new-name>
```

```
git remote rename origin upstream
```

Changes the name of the remote reference locally.

Advanced Git

41.What are Git hooks?

- Git hooks are scripts that run automatically at specific points in Git workflow to automate checks or tasks, like pre-commit or post-receive actions

42.What is a Git stash? When would you use it?

- Git stash **temporarily saves your uncommitted changes** (both staged and unstaged) so you can work on something else, then reapply them later.

Suppose you start on `branch1` with a file containing `abc`. You create two new branches, `b2` and `b3`. On `b2`, you edit the file to add `def` and `ghi` (so it contains `abc, def, ghi`) and then stash these changes using `git stash`. This temporarily saves your changes and restores the file to its last committed state, which is just `abc`. Next, you switch to `b3` and add `jkl` and `mno` to the file (so it now contains `abc, jkl, mno`). Later, you switch back to `b2`. At this point, the file on `b2` still shows only `abc` because the stashed changes have not been applied yet. To see the changes you stashed earlier (`def, ghi`), you need to run `git stash apply` (or `git stash pop`), which restores those changes on `b2`.

43. How do you apply and drop a stash?

- `git stash apply` // **Apply a stash without removing it from stash list**
- `git stash pop` // **Apply a stash and remove it from the stash list:**
- `git stash drop stash@{0}` // **Drop a specific stash without applying it:**
- `git stash clear` // **drop all**

44. How do you see all stashes in a repository?

- `git stash list`
- Shows all stashed changes in the repository.
- Each stash has an identifier like `stash@{0}`, `stash@{1}`, etc., along with the branch name and commit message at the time of stashing.
- `stash@{0}`: WIP on b2: abc123 added new feature
- `stash@{1}`: WIP on main: def456 fixed bug

45. What is a detached HEAD state in Git?

- HEAD is like a **pointer to the current branch's latest commit**.
- **Detached HEAD happens when...**

You check out a **specific commit** instead of a branch:

- `git checkout abc123`

- Now HEAD points directly to commit abc123, not a branch.
-
- Git says: “You are in detached HEAD state.”

▪ What it means

- You can **view or edit files** and even commit:


```
echo "new change" >> file.txt
git commit -m "experiment"
```
- But these commits **aren't on any branch**.
- If you switch to another branch, these commits can become **orphaned** (hard to find).

46.How do you recover a deleted branch in Git?

“If a branch is deleted, use `git reflog` to find its last commit, then run `git checkout -b <branch> <commit>` to restore it.”

- `git reflog` **records every change of HEAD** — not just commits, but also branch checkouts, resets, merges, and rebases.
- This means it keeps track of **all commits that your local repository has pointed to at any time**, even if:
 - The branch was deleted
 - The commit is no longer reachable from any branch
- It **doesn't store commits from other people's remotes**, only what your local repo has seen or pointed to.
- U deleteed a brnahc localaly: `git branch -d feature1`
- now **git reflog**
- Look for the commit hash of the deleted branch.

```
abc1234 HEAD@{0}: checkout: moving from feature1 to main
def5678 HEAD@{1}: commit: added new feature...
```

- Here, `def5678` is the **last commit on feature1** before deletion.
- `git checkout -b feature1 def5678`
- `git branch`
- `# main`
- `# feature1`

47. How do you find a commit that introduced a bug?

Steps

1. Start bisecting:

```
git bisect start
```

2. Mark the current commit as bad (contains the bug):

```
git bisect bad
```

3. Mark a known good commit (before the bug appeared):

```
git bisect good <commit-hash>
```

4. Git automatically checks out a commit in the middle of the range.

5. Test this commit:

- If the bug exists → `git bisect bad`
- If the bug does not exist → `git bisect good`

6. Repeat until Git identifies the **first bad commit** that introduced the bug.

7. End bisecting:

```
git bisect reset
```

48. What is `git cherry-pick` and when would you use it?

`git cherry-pick <commit-hash>` applies a specific commit from another branch onto your current branch.

When to use it

1. You want **only a specific bug fix or feature** from another branch without merging all changes.
2. You need to **apply an urgent fix from a development branch to main**.
3. Selectively move commits between branches in a **controlled manner**.

49. What is the difference between `git log`, `git reflog`, and `git show`?

Command	Purpose	What it shows	Typical Use Case
<code>git log</code>	Shows commit history	List of commits on the current branch (hash, author, date, message)	Reviewing commit history, finding when changes were made

Command	Purpose	What it shows	Typical Use Case
git reflog	Whenever head moves..(like commt or branch change).. trakcs all heads	Whenever head moves..(like commt or branch change).. local history of all head movements	Recovering lost commits or deleted branches
git show	Shows details of a specific commit	Commit details: author, date, message, and diff of changes in that commit	Inspecting exactly what a commit changed

50.How do you tag a commit in Git and push tags to remote?

```
git tag <tagname>          # Creates a lightweight tag on the latest commit
git tag -a <tagname> -m "msg" # Creates an annotated tag with a message
git tag -a <tagname> <commit-hash> -m "msg" # Annotated tag on a specific commit
git push origin <tagname>    # Push a specific tag to the remote repository
git push origin --tags      # Push all tags to the remote repository
```